

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ МИХАЙЛА
ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ
ІНЖЕНЕРІЇ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРА ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 8

виконав:
студент групи КН-25-1
Заїченко А.С.
Перевірив:
Доцент кафедри КІЕ
Сидоренко В.М.

Кременчук 2026

Практична робота № 8

Тема. Жадібні алгоритми. Наближене розв'язання екстремальних задач

Мета: набути практичних навичок застосування деяких жадібних алгоритмів для розв'язання екстремальних задач.

Постановка завдання

Розв'язати задачу комівояжера для зваженого графа: [(1,2,3), (1,3,5), (1,4,7), (2,3,6), (2,4,8), (3,4,9)]. Візуалізувати граф. Обґрунтувати асимптотику для обох алгоритмів.

Код на python:

```
INFINITY = pow(10, 20)
# Функція для обчислення довжини шляху
def get_path_length(path):
    path_length = 0
    # Проглядаємо усі вершини
    for i, v1 in enumerate(path):
        # Знаходимо наступну вершину
        v2 = path[(i + 1) % len(path)]
        # Перевіряємо, чи існує ребро
        if not G.has_edge(v1, v2):
            return INFINITY
        # Додаємо вагу ребра
        path_length += G[v1][v2]["weight"]
    return path_length
# Функція генерації перестановок
def node_permutations(G, init_node_index):
    nodes = list(G.nodes())
    # Початкова вершина
    init_node = nodes[init_node_index]
    # Видаляємо її
    nodes.remove(init_node)
    # Генеруємо перестановки
    return [[init_node] + list(a_tuple)
            for a_tuple in permutations(nodes)]
def TSP_BruteForce(G, init_node_index):
    min_path = min_path_length = None
```

```

# Перебір усіх шляхів
for path in node_permutations(G, init_node_index):
    # Довжина поточного шляху
    path_length = get_path_length(path)
    # Пошук мінімального
    if not min_path or min_path_length > path_length:
        min_path = path
        min_path_length = path_length
return min_path, min_path_length

```

Візуалізація графа:

```

G = nx.Graph()
G.add_nodes_from(range(1, 5))
G.add_weighted_edges_from([(1, 2, 3), (1, 3, 5), (1, 4, 7),
                           (2, 3, 6), (2, 4, 8), (3, 4, 9)])

plt.figure(figsize=(10, 7))
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G, pos, node_size=700)
nx.draw_networkx_labels(G, pos, font_size=20, font_family='sans-serif')
edges = [(u, v) for (u, v, d) in G.edges(data=True)]
nx.draw_networkx_edges(G, pos, edgelist=edges)
edge_labels = dict([(u, v), d['weight']]
                    for u, v, d in G.edges(data=True))
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
show() #вивести графік

```

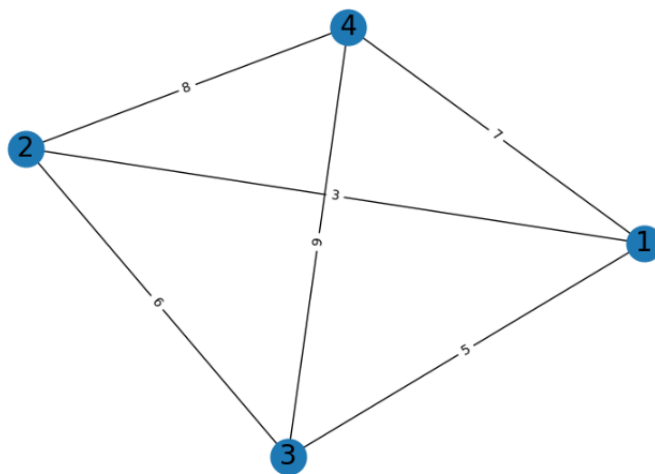


Рисунок 1 - Візуалізація графа

Найкоротший маршрут: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$

Мінімальна довжина шляху: 25

Асимптотика:

1. Алгоритм повного перебору
 - Кількість можливих маршрутів: $(n - 1)!$
 - Асимптотична складність: $O(n!)$
2. Алгоритм найближчого сусіда
 - На кожному кроці переглядаються всі невідвідані вершини.
 - Асимптотична складність: $O(n^2)$

Контрольні питання

1. Що таке жадібний алгоритм?

Це метод проєктування алгоритмів, який полягає в тому, що на кожному кроці приймається локально оптимальне рішення у надії, що фінальний результат також буде глобально оптимальним.

2. Які головні принципи роботи жадібних алгоритмів?

- Принцип жадібного вибору
- Властивість оптимальної підструктури

3. Яка головна відмінність між жадібними алгоритмами та динамічним програмуванням?

Жадібний алгоритм робить вибір лише один раз на кожному кроці і ніколи не скасовує своїх рішень.

Динамічне програмування розбиває задачу на підзадачі, розв'язує їх, зберігає результати в таблиці та розглядає всі можливі комбінації попередніх рішень для пошуку абсолютно найкращого результату.

4. Наведіть приклади задач, які можна розв'язати за допомогою жадібних алгоритмів.

- Побудова мінімального кістякового дерева
- Пошук найкоротшого шляху у графі
- Кодування Гаффмана

5. Які можуть бути обмеження у використанні жадібних алгоритмів для розв'язання екстремальних задач?

1) Локальний вибір може завести алгоритм у глухий кут або змусити згодом зробити вкрай не вигідні кроки.

2) Якщо задача не має властивості жадібного вибору, алгоритм видасть неправильну відповідь.

6. Чому жадібні алгоритми часто використовуються для наближеного розв'язання екстремальних задач?

1) Вони зазвичай працюють за лінійний або поліноміальний час. Це дозволяє миттєво обробляти мільйони елементів там, де точний алгоритм рахував би століттями.

2) Для багатьох інженерних та бізнес-задач розв'язок, який відхиляється від ідеального всього на кілька відсотків, але знаходиться за мілісекунди, є набагато ціннішим, ніж ідеальний розв'язок, отриманий занадто пізно.